

1 Fonaments de Programació

1.1 Unitat 2: Breu introducció a la programació en Python

1.1.1 Instruccions d'ús

Aquest document és un *notebook* interactiu que intercala explicacions més aviat teòriques de conceptes de programació amb fragments de codi executables. Per aprofitar els avantatges que aporta aquest format, us recomanem que, en primer lloc, llegiu les explicacions i el codi que us proporcionem. D'aquesta manera tindreu un primer contacte amb els conceptes que hi exposem. Ara bé, **la lectura és només el principi!** Una vegada hàgiu llegit el contingut, no oblideu executar el codi proporcionat i modificar-lo per crear-ne variants que us permetin comprovar que heu entès la seva funcionalitat i explorar-ne els detalls d'implementació. Per últim, us recomanem també consultar la documentació enllaçada per explorar amb més profunditat les funcionalitats dels mòduls presentats.

1.1.2 Introducció

En aquesta unitat presentarem el llenguatge de programació Python i la seva sintaxi bàsica. Explicarem què són les variables, veurem com les podem definir i utilitzar en Python, i n'introduïrem els principals tipus simples (enters, floats, complexos) i compostos (cadena de caràcters, llistes, tuples i diccionaris).

A continuació s'inclou la taula de continguts, que podeu fer servir per navegar pel document:

1. Què és la programació?
2. Breu història del llenguatge de programació Python
3. Versions i intèrprets disponibles
4. Sintaxi bàsica del llenguatge de programació Python
- 4.1. Intèrpret enfront d'script

4.2. Sobre els tabuladors i els espais en blanc

4.3. Comentaris

4.4. Guia d'estil

4.5. IPython i notebook

```
<li><a href="#4.6.-Colab">4.6. Colab</a></li>
</ul>
<li><a href="#5.-Variables-i-tipus-de-variables">5. Variables i tipus de variables</a></li>
  <ul style="list-style-type:none">
    <li><a href="#5.1.-Declaració-de-variables">5.1. Declaració de variables</a></li>
    <li><a href="#5.2.-Paraules-reservades">5.2. Paraules reservades</a></li>
    <li><a href="#5.3.-Funcions-bàsiques">5.3. Funcions bàsiques</a></li>
      <li><a href="#5.3.1-Funció-print">5.3.1 Funció print</a></li>
      <li><a href="#5.3.2-Funció-input">5.3.2 Funció input</a></li>
    <li><a href="#5.4.-Tipus-de-dades">5.4. Tipus de dades</a></li>
      <ul style="list-style-type:none">
        <li><a href="#5.4.1-Tipus-numèrics-i-cadenes-de-caràcters">5.4.1 Tipus numèrics i cadenes de caràcters</a></li>
        <li><a href="#5.4.2-Llistes,-tuples-i-diccionaris">5.4.2 Llistes, tuples i diccionaris</a></li>
      </ul>
    </li>
  </ul>
</ul>
<li><a href="#6.-Organització-del-codi">6. Organització del codi</a></li>
<li><a href="#7.-Operadors">7. Operadors</a></li>
<ul style="list-style-type:none">
  <li><a href="#7.1.-Operadors-aritmètics">7.1. Operadors aritmètics</a></li>
  <li><a href="#7.2.-Operadors-d'assignació">7.2. Operadors d'assignació</a></li>
  <li><a href="#7.3.-Operadors-relacionals">7.3. Operadors relacionals</a></li>
  <li><a href="#7.4.-Operadors-lògics">7.4. Operadors lògics</a></li>
  <li><a href="#7.5.-Operadors-de-pertinença">7.5. Operadors de pertinença</a></li>
  <li><a href="#7.6.-Numpy">7.6. Numpy</a></li>
</ul>
<li><a href="#8.-Recursos-per-al-programador-de-Python">8. Recursos per al programador de Python</a></li>
<li><a href="#9.-Exercicis-i-preguntes-teòriques">9. Exercicis i preguntes teòriques</a></li>
  <ul style="list-style-type:none">
    <li><a href="#9.1-Instruccions-importants">9.1 Instruccions importants</a></li>
  </ul>
</ul>
<li><a href="#10.-Bibliografia">10. Bibliografia</a></li>
```

2 1. Què és la programació?

Troblem a Wikipedia la definició següent de programació:

«Programació d'ordinadors o **programació informàtica** (sovint abreujat programació o codificació) és el procés d'escriure, provar, **depurar** i mantenir el **codi font** de **programes**. Aquest codi font està escrit en un **llenguatge de programació**. L'objectiu de la programació és crear un programa que mostra un determinat comportament desitjat.»

En el context de la ciència de dades, els ordinadors són màquines que ens permeten tractar, processar i analitzar quantitats ingents de dades. Aquest tractament de la informació ha de realitzar-se mitjançant un tipus de conjunt de regles: l'**algorisme**. Podem entendre els algorismes com a plantilles o «receptes» que s'especifiquen a la màquina mitjançant algun llenguatge de programació i que ens permeten tractar la informació de forma inherent a les nostres necessitats. Seguint amb la metàfora de la recepta, imagineu que volem preparar un plat de cuina i que tenim tots els ingredients per fer-ho. La nostra recepta culinària especificarà en cada pas el que el cuiner ha de fer. Per exemple, per fer una truita:

1. Trencar els ous i batre'ls.
2. Posar una paella al foc.
3. Quan la temperatura estigui prou alta, afegir-hi els ous batuts.
4. Esperar que qualli suficientment.
5. Donar la volta.
6. Repetir el pas 4 per aquesta cara.
7. Posar la nostra truita al plat.

Els problemes que resolldrem mitjançant un ordinador normalment tindran un context més numèric, però la idea de recepta serà la mateixa: un conjunt d'instruccions que s'executen des del principi fins al final i que duen a terme una acció. Així doncs, l'algorisme o «recepta» per calcular el quadrat d'un nombre qualsevol seria de la manera següent:

1. Representar a la pantalla el missatge «Benvingut a la calculadora de quadrats:».
2. Demanar a l'usuari que introdueixi un nombre mitjançant el teclat.
3. Guardar aquest nombre.
4. Prendre el nombre guardat com a base per multiplicar-lo per si mateix.
5. Representar en pantalla el resultat del càlcul del pas 4.
6. Sortir del programa.

Especificar-li a un ordinador les instruccions que han d'executar-se en un programa és una abstracció que ens permeten els llenguatges de programació. En el nostre cas, farem servir Python per escriure i executar els nostres programes i així solucionar problemes de ciència de dades.

3 2. Breu història del llenguatge de programació Python

El llenguatge va néixer de la mà de [Guido van Rossum](#) mentre treballava al Centre per a les Matemàtiques i la Informàtica (CWI, Centrum Wiskunde & Informatica) als Països Baixos l'any 1989 durant les vacances de Nadal, encara que no va ser fins al febrer de 1991 quan va ser publicada la seva primera versió a [USENET](#).

Guido van Rossum és, malgrat el llarg recorregut del llenguatge des d'aleshores, una figura molt rellevant en el grup d'usuaris i desenvolupadors de Python i és conegut com BDFL (de l'anglès *benevolent dictator for life*, dictador vitalici benevolent); els seus suggeriments en les discussions en les quals intervé es tenen molt en compte respecte al full de ruta del futur del llenguatge. Actualment, la *Python Software Foundation* (PSF) és l'organització encarregada de vetllar pel futur i la disseminació del llenguatge. Com a nota divertida, cal destacar que la relació entre el nom del llenguatge i la sèrie de televisió [Monty Python's Flying Circus](#) no és una mera coincidència.

Hi ha dues dates importants que han marcat la història del llenguatge: el 16 d'octubre de 2000,

moment en el qual la versió 2.0 va ser publicada, i el 3 de desembre de 2008, quan es va publicar la versió 3.0 del llenguatge.

El llenguatge de programació Python fa especial èmfasi en la seva capacitat perquè altres programadors el puguin llegir. Aquesta senzillesa aparent a l'hora de llegir codi en Python li ha permès gaudir de bastanta popularitat en cercles menys enfocats en el desenvolupament pur de programari, com pot ser el camp de la ciència de dades, en el qual conflueixen persones expertes en diferents disciplines (matemàtiques, estadística, etc.). Existeixen documents de millora del llenguatge coneguts per les seves sigles **PEP-Número** (de l'anglès *Python Enhancement Proposals*) i és, precisament, en un d'aquests documents, el [PEP-20](#), en el qual es presenta la filosofia del llenguatge i aquesta marcada tendència per la simplicitat, llegibilitat i bellesa del codi en Python:

The Zen of Python

```
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one -and preferably only one- obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -let's do more of those!-
```

Molts d'aquests principis en aquest moment podran semblar estranys a l'alumnat, però a mesura que s'aprofundeix en l'ús del llenguatge molts d'ells resultaran habituals a l'hora d'escriure i llegir codi en Python.

Finalment, és interessant presentar a l'alumnat el terme *pythonic*, ja que apareix en moltes cerques en relació amb la sintaxi de Python. Els llenguatges de programació fan ús habitualment d'estructures que serveixen per expressar tasques o parts de codi en un llenguatge comú que no està considerat directament en la sintaxi del llenguatge. Aquest concepte (en anglès *idiom*) està molt estès en la comunitat de Python i moltes vegades es tradueix en missatges en fòrums de l'estil «*which is the most pythonic way to do..?*» ('quina és la forma més pythònica de fer..?'). Es tracta d'un concepte avançat, però que en Python resulta especialment rellevant, atès que alguns d'aquests conceptes han estat implementats en el llenguatge de forma més eficient i poden fer que un codi o un altre s'executi de forma més ràpida si és més pythònica. Aquest concepte serà ampliat més endavant.

4 3. Versions i intèrprets disponibles

Actualment coexisteixen dues versions o branques de desenvolupament: les famílies 2.7 i 3.6, sent les seves últimes versions al moment de redacció d'aquest document la 2.7.18 i la 3.10.1, respectivament.

Python és un llenguatge de programació molt viu i en contínua evolució, al qual es van afegint i corregint funcionalitats molt sovint. La branca de desenvolupament 2.7.x és una versió molt madura i estable, però va ser amb l'aparició de la versió 3.0 quan va haver-hi un salt important en la sintaxi del llenguatge. Per exemple, la mateixa expressió `print` en la branca 2.7.x va passar a tenir forma de funció `print()` en la branca 3.x. Aquests canvis no es limiten a correccions estètiques, sinó que s'han introduït per millorar el rendiment del llenguatge, l'organització del codi i de les llibreries a nivell intern, etc. En resum, la branca 3.x és el futur de Python mentre que la branca 2.7.x representaria el passat del llenguatge.

Al Gener de 2020, va finalitzar el suport de la branca 2.x de Python, i, actualment, la família que es fa servir és la 3.x. Per aquest motiu, en aquest curs utilitzem Python 3.x. Tot i això, si mai heu de treballar amb codi antic desenvolupat per Python 2.x, veureu que no tindreu problemes per fer-ho, ja que l'essència del llenguatge segueix sent la mateixa.

Quant als intèrprets, el més conegut és la implementació en C anomenada [CPython](#), que és l'intèrpret del llenguatge Python desenvolupat per la comunitat del projecte de codi lliure i suportat i dirigit per la Python Programari Foundation. Encara que CPython és la implementació estàndard i a dia d'avui suporta més plataformes (quaranta-dues en el moment de redacció d'aquest document), no és ni molt menys l'única. Altres implementacions famoses són [Jython](#), escrit en Java i que funciona en la màquina virtual de Java; [PyPy](#), un intèrpret també escrit en C, però que posa especial èmfasi en el rendiment i a tractar la concurrència i el paral·lelisme d'una forma diferent; [IronPython](#), una implementació del llenguatge per a la plataforma .NET, etc.

5 4. Sintaxi bàsica del llenguatge de programació Python

5.1 4.1. Intèrpret enfront d'script

Hi ha tres formes bàsiques d'executar codi en Python:

1. Executar l'intèrpret de Python des d'una terminal.
2. Escriure codi Python en un fitxer (típicament amb extensió PY, i que és conegut habitualment amb el nom d'script) i executar-lo utilitzant l'ordre:

```
$ python script_name.py
```

3. Escriure codi Python mitjançant *Jupyter Notebook*.

La primera forma és útil per provar ordres del llenguatge Python de forma ràpida o simplement per fer servir l'intèrpret com a calculadora. La segona, per contra, ens permetrà executar una vegada i una altra el codi contingut en el nostre fitxer d'script sense haver d'escriure en l'intèrpret el mateix codi una vegada i una altra. La tercera opció és la que farem servir en aquesta assignatura i l'explicarem amb més detall a l'apartat 4.5.

5.2 4.2. Sobre els tabuladors i els espais en blanc

En Python, els blocs de codi s'indiquen mitjançant espais en blanc. Més endavant parlarem dels diferents tipus de blocs de codi, però a manera introductòria farem ús de l'exemple següent:

```
1 def suma(a, b):  
2     return a + b  
3  
4 print(suma(2, 3))
```

En les línies 1 i 2, definim mitjançant la paraula clau **def** una funció que rep com a arguments dues variables de nom **a** i **b** i que a la línia 2 retorna la suma d'aquestes dues variables.

En la línia 4, cridem aquesta funció suma amb els valors 2 i 3, i mitjançant l'ordre **print** escrivim el resultat que ens apareix per la terminal.

Com es pot observar en la línia 2, la paraula clau **return** es troba a quatre espais de distància del començament de la línia anterior (línia 1). Això és així perquè estem definint un bloc de codi (en altres llenguatges com C o Java, s'utilitzen per a la definició de blocs de codi els caràcters especials «{» i «}»).

Existeixen dues possibilitats per definir la indentació de codi (indentació és un anglicisme i en català hauríem d'utilitzar el terme sagnat, encara que indentació està àmpliament reconeguda per la comunitat informàtica): mitjançant espais o tabulacions. Podrem fer servir de forma indistinta una forma o una altra, però sempre tenint en compte que mai no hauran de barrejar-se en un mateix fitxer de codi. Això és així perquè les tabulacions no tenen un caràcter que les distingeixi i no solen presentar-se de forma idònia en molts editors de text. Moltes vegades, en pressionar la tecla «Tab» en una aplicació, aquest caràcter es converteix automàticament en un nombre fix d'espais en blanc. Per tant, **serà recomanable configurar l'editor que escollim per escriure codi en Python de tal forma que el tabulador es tradueixi en quatre espais en blanc.**

En el document [PEP-8](#), que és la guia d'estil que segueixen la majoria de programadors en Python, es detalla aquest aspecte.

5.3 4.3. Comentaris

Els comentaris, fragments de text que no s'interpreten, són una part fonamental de qualsevol programa, ja que faciliten la lectura i permeten afegir aclaracions necessàries per a la correcta comprensió del codi.

Si no s'afegeixen comentaris al codi, pot ser que l'usuari no recordi algunes de les decisions que va prendre quan el va escriure. D'altra banda, si es llegeix codi escrit per una altra persona, els comentaris permeten seguir millor els raonaments darrere d'aquest codi.

En Python, es poden incloure comentaris en el codi d'una sola línia (precedits pel símbol **#**) o bé de múltiples línies (que s'indiquen amb **"""** o bé **'''**). Els comentaris de múltiples línies s'acostumen a utilitzar per a documentar funcions, mètodes o classes en Python.

```
[1]: # Això és un comentari. Quan executem aquesta cel·la  
    # el comentari s'ignora.
```

```
[2]: def suma(a,b):  
    """ Aquesta funció retorna  
    la suma dels arguments a i b """  
    return a + b
```

5.4 4.4. Guia d'estil

A l'apartat anterior s'ha tractat la guia d'estil oficial del llenguatge Python, que està disponible en línia com PEP-8. Com a resum, aquests són els aspectes bàsics que s'han de seguir:

1. Les funcions i els noms de paraules s'escriuran en minúscula i els noms llargs que continguin diverses paraules s'uniran mitjançant el caràcter «_».
2. Els noms de classes tindran la seva lletra inicial en majúscula per diferenciar-les de variables, funcions i objectes.
3. Les constants aniran en majúscules i separades pel caràcter «_» si contenen diverses paraules.
4. Com ja s'ha comentat a l'apartat anterior, no s'utilitzaran tabuladors, sinó que sempre farem servir quatre espais en blanc.
5. Els comentaris han de ser frases completes i han d'estar actualitzats (és important que corresponguin al codi que s'està comentant!).

És molt important seguir la guia d'estil quan es programa en Python (recordeu-ho quan prepareu el codi per als lliuraments de l'assignatura!).

5.5 4.5. IPython i Notebook

IPython va ser originalment desenvolupat com un conjunt d'eines per facilitar el desenvolupament i el processament de dades en Python a usuaris del món científic. La «i» del nom prové de interactiu, ja que disposa d'eines que interactuen amb l'usuari, com són l'autocompletat de codi, una ajuda extensa, etc.

El **Jupyter Notebook**, al seu torn, és una altra eina molt útil per a l'exploració de codi, per representar resultats de forma gràfica, o pel prototipatge ràpid, etc., tot això mitjançant la interacció via navegador web (Notebook arrenca de forma local un servidor web). Les versions anteriors de Jupyter Notebook es coneixien com IPython Notebook.

Un notebook s'organitza en cel·les, que segueixen un ordre seqüencial. Hi ha 3 tipus de cel·les, però s'utilitzen principalment, dos: **text** (anomenat *markdown*) i **codi**. Ambdós tipus de cel·les es poden executar, però tenen funcionaments diferents. Una cel·la de tipus codi s'utilitza per executar el bloc de codi de Python que estigui dintre aquesta cel·la. D'altra banda, una cel·la *markdown* ens permet donar format al text (per exemple, es poden posar elements en **negreta** o *cursiva*) o afegir imatges al notebook. El tercer tipus de cel·la s'anomena *Raw NBConvert* i s'utilitza per posar codi que no es vulgui executar, ja que aquestes cel·les no són evaluades pel notebook. El tipus de la cel·la no és fixe. Un cop creada, es pot canviar el tipus d'una cel·la (per exemple, pots convertir una cel·la de codi a text, i viceversa).

Així doncs, els notebooks ens proporciona un ambient molt dinàmic, ja que podem escriure i evaluar el nostre codi fàcilment. També ens permet combinar blocs de text i blocs de codi, cosa que el converteix en una eina que molt adequada en l'àmbit formatiu.

5.6 4.6. Colab

Actualment, el desenvolupament de codi es pot fer també mitjançant plataformes al núvol. La computació al núvol (*cloud computing*) permet tenir accés remot a programes, emmagatzemar arxius i processar grans volums de dades, sense tenir les limitacions d'una màquina local. Una plataforma molt coneguda i utilitzada és el **Google Colab**, un entorn que ens permet escriure i executar codi Python mitjançant notebooks completament des del núvol. El seu ús ha anat en augment en els últims anys, ja que permet configurar determinades prestacions de l'equip que són difícils de tenir a nivell local (per exemple, es pot activar la GPU si es necessita un alt poder de computació). D'altra banda, a l'executar-se des del núvol, també permet compartir fàcilment els notebooks quan es treballa en equip.

En aquesta assignatura farem servir Google Colab, ja que ens proporciona un entorn comú i col·laboratiu entre tots els estudiants.

6 5. Variables i tipus de variables

Podem entendre una **variable** com un contenidor en el qual podem posar les nostres dades a fi de guardar-les i tractar-les més endavant. A Python, les variables no tenen tipus, és a dir, no hem d'indicar si la variable serà numèrica, un caràcter, una cadena de caràcters o una llista, per exemple. A més, les variables poden ser declarades i inicialitzades en qualsevol moment, a diferència d'altres llenguatges de programació.

6.1 5.1. Declaració de variables

Per **declarar** una variable, fem servir l'expressió *nom_de_variable = valor*. Es recomana repassar el document [PEP-8](#) per indicar noms de variables correctes, però, *grosso modo*, evitarem utilitzar majúscula a la inicial, separarem les diferents paraules amb el caràcter «`_`» i no utilitzarem accents ni caràcters específics de la nostra codificació com el símbol del «`€`» o la «`ñ`», per exemple.

Vegem uns quants exemples de declaracions de variables i com fer-les servir:

```
[3]: # Declarem una variable de nom 'variable_numerica'
# que conté el valor enter 12.
variable_numerica = 12

# Declarem una variable de nom monstre
# que conté el valor 'Godzilla'.
monstre = 'Godzilla'

# Declarem una variable de nom 'planetes'
# que és una llista de cadenes de caràcters.
planetes = ['Mercuri', 'Venus', 'Terra', 'Mart']
```

```
[4]: la_meva_edat = 25
la_meva_edat_en_5 = la_meva_edat + 5
```



```
# 'Imprimim' el valor calculat que serà, efectivament, 30
print(la_meva_edat_en_5)
```

30

6.2 5.2. Paraules reservades

En Python, hi ha un conjunt de paraules, anomenades **paraules reservades**, que no podem utilitzar per declarar variables, ja que tenen un significat especial i el seu nom està reservat per certes tasques internes del llenguatge de programació.

Algunes d'aquestes paraules formen part de la sintaxi bàsica de Python (per exemple: True, False, def, return, for, if). D'altres formen part de funcionalitats més avançades (async, await).

Per consultar quines són les paraules reservades en Python, es pot executar el codi següent:

```
[5]: help('keywords')
```

Here is a list of the Python keywords. Enter any keyword to get more help.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

6.3 5.3. Funcions bàsiques

Una part clau en el disseny és que el codi sigui dinàmic i interactiu per l'usuari. Per aconseguir-ho, hi ha funcions bàsiques de Python que permeten a l'usuari interaccionar amb el programa.

Una **funció** és una porció de codi reutilitzable que s'utilitza per a fer una tasca determinada. Si tenim un tros de codi que hem d'utilitzar repetidament, podem utilitzar la funció que conté el codi dintre, enlloc d'haver d'escriure'l tot un altre cop.

A continuació teniu l'exemple de la funció suma que havíem creat anteriorment:

```
[6]: # Definim la funció suma
def suma(a, b):
    """ Aquesta funció retorna
    la suma dels arguments a i b """
    return a + b
```

```
[7]: # Si volem fer la suma de dos nombres,  
# no cal escriure tot el codi un altre cop  
suma(1, 2)
```

[7]: 3

A la unitat 3 veurem en detall les funcions, però, de moment, és important conèixer dues funcions bàsiques: la funció `print` i la funció `input`.

6.3.1 5.3.1. Funció print

La funció `print()` agafa com a paràmetre d'entrada un o més elements i els mostra per pantalla. Al imprimir-se, els elements es transformen en format text. Si hi ha més d'un paràmetre, aquests es mostren per defecte separats per un espai i, al final, s'acaba amb un salt de línia.

```
[8]: # Mostrem per pantalla 3 elements  
print(1, 2, 3)  
  
# Mostrem per pantalla dos elements de text  
print('Hello', 'world')  
  
# Podem també escriure fragments de text  
print('Hello world')  
  
# Si no posem cap paràmetre, print retorna un salt de línia  
print()  
  
# També podem especificar altres separadors enlloc de l'espai  
print(1, 2, 3, sep = ':')
```

```
1 2 3  
Hello world  
Hello world
```

```
1:2:3
```

Un altre aspecte a destacar són les **seqüències d'escapament** (en anglès, *escape sequences*), conjunts de caràcters que tenen un significat especial per l'interpret. En Python, quan utilitzem la funció `print()`, podem especificar els salts de línia o el tabulador mitjançant seqüències de fugida. Per exemple:

```
[9]: # Utilitzem \t per a afegir un tabulador  
print('Hello\tworld')  
  
# Utilitzem \n per a afegir un salt de línia  
print('Hello\nworld')
```

```
Hello    world
Hello
world
```

6.3.2 5.3.2. Funció input

La funció **input** permet a l'usuari introduir un valor per pantalla que, posteriorment, pot ser guardat i utilitzat pel programa. És important destacar que l'element que l'usuari especifica mitjançant `input()` es retorna en forma de text.

Aquesta funció és molt útil ja que permet la interacció entre el codi i el seu usuari.

```
[10]: # En aquest exemple, demanem a l'usuari que introdueixi el seu nom
# Aquesta informació es guarda a la variable "nom"
# Aquí també podem veure com utilitzem \n per a fer el salt de línia
nom = input("Introdueix el teu nom i cognoms\n")

# La variable "nom" pot ser utilitzada després
print("El nom introduït és el següent: ", nom)
```

```
Introdueix el teu nom i cognoms
Alex Ortega
El nom introduït és el següent:  Alex Ortega
```

6.4 5.4. Tipus de dades

A continuació presentem alguns dels tipus nadius de dades que una variable de Python pot contenir: nombres enters (int), nombres decimals (float), nombres complexos (complex), cadena de caràcters (string), llistes (list), tuples (tuple) i diccionaris (dict).

Tipus de dada	Exemple
enters (int)	num_enter = 1
nombres decimals (float)	num_dec = 1.5
nombres complexos (complex)	num_complex = 2 + 3j
cadena de caràcters (string)	text = "Hello World"
llistes (list)	llista = [1, 2, 3]
tuples (tuple)	pel·li = ('Aladdin', 1992)
diccionaris (dict)	fruita = {'pera': 1.5, 'poma': 3.4}

Vegem uns exemples per un cada un d'aquests tipus:

6.4.1 5.4.1. Tipus numèrics i cadenes de caràcters

```
[11]: # Un nombre enter
int_var = 1
```

```

another_int_var = -5

# Podem sumar-los, restar-los, multiplicar-los o dividir-los.
print(int_var + another_int_var)
print(int_var - another_int_var)
print(int_var * another_int_var)
print(int_var / another_int_var)

# També podem realitzar la divisió entera.
# Com que només tractem amb nombres enters, no hi haurà part decimal.
print(int_var // another_int_var)

```

```

-4
6
-5
-0.2
-1

```

El comportament de l'operador / és una de les diferències entre Python 2 i Python 3. Mentre que en Python 3, l'operador / realitza la divisió real entre dos nombres enters (fixeu-vos que `1 / -5` dóna com a resultat `0.2`), en Python 2 realitzava la divisió entera (per la qual cosa el resultat d'executar `1 / -5` en Python 2 seria `-1`). Noteu que usem l'operador `//` per expressar la divisió entera en Python 3.

```

[12]: # Un nombre decimal o 'float'
float_var = 2.5
another_float_var = .7

# Convertim un nombre enter en un de decimal
# mitjançant la funció 'float()'
encore_float = float(7)

# Podem fer el mateix en sentit contrari
# amb la funció 'int()'
new_int = int(encore_float)

# Podem fer les mateixes operacions que
# en el cas dels nombres enters,
# però en aquest cas la divisió serà
# decimal si algun dels nombres és decimal.
print(another_float_var + float_var)
print(another_float_var - float_var)
print(another_float_var * float_var)
print(another_float_var / float_var)
print(another_float_var // float_var)

```

```

3.2
-1.8

```

```
1.75
0.27999999999999997
0.0
```

```
[13]: # Un nombre complex
complex_var = 2 + 3j

# Podem accedir a la part imaginària o a la part real:
print(complex_var.imag)
print(complex_var.real)
```

```
3.0
2.0
```

```
[14]: # Cadena de caràcters
my_string = 'Hello, Bio! ñç'
print(my_string)
```

```
Hello, Bio! ñç
```

Fixeu-vos que podem incloure caràcters unicode (com ñç) a les cadenes. Això també és una novetat de Python 3 (les variables de tipus `str` són ara UTF-8).

```
[15]: # Podem concatenar dues cadenes utilitzant l'operador '+'.
same_string = 'Hello, ' + 'Bio' + '!' + ' ñç'
print(same_string)

# A Python també podem utilitzar wildcards
# com en la funció 'sprintf' de C.
# Per exemple:
name = "Guido"
num_emails = 5
print("Hello, %s! You've got %d new emails" % (name, num_emails))
```

```
Hello, Bio! ñç
```

```
Hello, Guido! You've got 5 new emails
```

A l'exemple anterior, hem substituït a l'`string` la cadena `%s` pel contingut de la variable `name`, que és un `string`, i `%d` per `num_emails`, que és un nombre enter. També podríem utilitzar `%f` per a nombres decimals (podríem indicar la precisió, per exemple, amb `%5.3f`, el nombre tindria una mida total de cinc xifres i tres serien per la part decimal). Hi ha moltes altres possibilitats, però haurem de tenir en compte el tipus de variable que volem substituir. Per exemple, si utilitzem `%d` i el contingut és `string`, Python retornarà un missatge d'error. Per evitar aquesta situació, es recomana l'ús de la funció `str()` per convertir el valor a `string`.

També podem mostrar el contingut de les variables sense especificar el seu tipus, fent servir `format`:

```
[16]: print("Hello, {}! You've got {} new emails".format(name, num_emails))
```

```
Hello, Guido! You've got 5 new emails
```

6.4.2 5.4.2. Llistes, tuples i diccionaris

Ara presentarem altres tipus de dades nadius més complexos: llistes, tuples i diccionaris:

```
[17]: # Definim una llista amb el nom dels planetes (string).
planets = ['Mercury', 'Venus', 'Earth', 'Mars',
           'Jupiter', 'Saturn', 'Uranus', 'Neptune']

# També pot contenir números.
prime_numbers = [2, 3, 5, 7]

# Una llista buida
empty_list = []

# O una barreja de qualsevol tipus:
sandbox = ['3', 'a string',
           ['a list inside another list', 'second item'],
           7.5]

# Mostrem els elements per pantalla
print(sandbox)
```

```
['3', 'a string', ['a list inside another list', 'second item'], 7.5]
```

```
[18]: # Podem afegir elements a una llista.
planets.append('Pluto')
print(planets)
```

```
['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune',
'Pluto']
```

```
[19]: # O en podem eliminar.
planets.remove('Pluto')
print(planets)
```

```
['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune']
```

```
[20]: # Podem eliminar qualsevol element de la llista.
planets.remove('Venus')
print(planets)
```

```
['Mercury', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune']
```

```
[21]: # Sempre que n'afegim, serà al final de la llista.
# Una llista està ordenada.
planets.append('Venus')
print(planets)
```

```
['Mercury', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune', 'Venus']
```

```
[22]: # Si volem ordenar-la alfabèticament,  
# podem utilitzar la funció 'sorted()'  
print(sorted(planets))
```

['Earth', 'Jupiter', 'Mars', 'Mercury', 'Neptune', 'Saturn', 'Uranus', 'Venus']

```
[23]: # Podem concatenar dues llistes:  
monsters = ['Godzilla', 'King Kong']  
more_monsters = ['Cthulu']  
  
# Mostrem el resultat per pantalla  
print(monsters + more_monsters)
```

['Godzilla', 'King Kong', 'Cthulu']

```
[24]: # Podem concatenar una llista amb una altra  
# i guardar-la a la mateixa llista:  
monsters.extend(more_monsters)  
  
# Mostrem el resultat per pantalla  
print(monsters)
```

['Godzilla', 'King Kong', 'Cthulu']

```
[25]: # Podem accedir a un element en concret de la llista:  
print(monsters[0])  
  
# El primer element d'una llista és el 0, per tant, el segon serà l'1:  
print(monsters[1])  
  
# Podem accedir a l'últim element mitjançant nombres negatius:  
print(monsters[-1])  
  
# Penúltim:  
print(monsters[-2])
```

Godzilla
King Kong
Cthulu
King Kong

```
[26]: # També podem obtenir parts d'una llista  
# mitjançant la tècnica de 'slicing'.  
planets = ['Mercury', 'Venus', 'Earth', 'Mars',  
           'Jupiter', 'Saturn', 'Uranus', 'Neptune']  
  
# Per exemple, els dos primers elements:  
print(planets[:2])
```

```
['Mercury', 'Venus']
```

```
[27]: # 0 els elements entre les posicions 2 i 4
      print(planets[2:5])
```

```
['Earth', 'Mars', 'Jupiter']
```

Fixeu-vos en aquest últim exemple: en la posició 2 trobem el tercer element de la llista ('Earth') ja que la llista comença a indexar en 0. A més, l'últim element indicat (la posició 5) no s'inclou.

```
[28]: # 0 els elements del segon al penúltim:
      print(planets[1:-1])
```

```
['Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus']
```

La tècnica de *slicing* és molt important i ens permet gestionar llistes d'una manera molt senzilla i potent. Serà imprescindible dominar-la per afrontar molts dels problemes que haurem de resoldre en el camp de la ciència de dades.

```
[29]: # Podem modificar un element en concret d'una llista:
      monsters = ['Godzilla', 'King Kong', 'Cthulu']
      monsters[-1] = 'Kraken'

      # Mostrem la llista per pantalla
      print(monsters)
```

```
['Godzilla', 'King Kong', 'Kraken']
```

```
[30]: # Una tupla és un tipus molt semblant a una llista,
      # però és immutable, és a dir, un cop declarada
      # no podem afegir-hi elements ni eliminar-ne:
      birth_year = ('Stephen Hawking', 1942)

      # Si executem la línia següent, obtindrem un error de tipus 'TypeError'
      birth_year[1] = 1984
```

```
↳ -----
```

TypeError

Traceback (most recent call last)

```
<ipython-input-30-bd7a85c84205> in <module>
```

```
5
```

```
6 # Si executem la línia següent, obtindrem un error de tipus 'TypeError'
----> 7 birth_year[1] = 1984
```

TypeError: 'tuple' object does not support item assignment

Els errors en Python solen ser molt informatius. Una recerca a internet ens ajudarà en la gran majoria de problemes que puguem tenir.

```
[31]: # Un string també és considerat una llista de caràcters.
# Així doncs, podem accedir a una posició determinada
# (tot i que no modificar-la):
name = 'Albert Einstein'
print(name[5])

# Podem fer servir slicing també amb les cadenas de caràcters
print(name[7:15])

# Podem separar pel caràcter que considerem un string.
# En aquest cas, per l'espai en blanc, utilitzant
# la funció 'split()'.
n, surname = name.split()
print(surname)

# I podem convertir un determinat string en una llista
# de caràcters fàcilment:
chars = list(surname)
print(chars)

# Per unir els diferents elements d'una llista
# mitjançant un caràcter, podem utilitzar la funció
# 'join()':
print(''.join(chars))
print('.'.join(chars))
```

```
t
Einstein
Einstein
['E', 'i', 'n', 's', 't', 'e', 'i', 'n']
Einstein
E.i.n.s.t.e.i.n
```

```
[32]: # L'operador ',' és el creador de tuples.
# Per exemple, el típic problema d'assignar el valor d'una
# variable a una altra en Python es pot resoldre
# en una línia d'una manera molt elegant utilitzant
# tuples (es tracta d'un idiom):
a = 5
b = -5
a, b = b, a
print(a)
print(b)
```

-5

5

L'anterior exemple és un *idiom* típic de Python. A la tercera línia, creem una tupla (a,b) a la qual assignem els valors un per un de la tupla (b,a). Els parèntesis no són necessaris, i per això queda una notació tan reduïda.

Per acabar, presentarem els diccionaris, una estructura de dades molt útil a la qual assignem un valor a una clau en el diccionari:

```
[33]: # Codis internacionals d'alguns països.  
# La clau o 'key' és el codi de país, i el valor, el seu nom:  
country_codes = {34: 'Spain', 376: 'Andorra',  
                 41: 'Switzerland', 424: None}  
  
# Podem buscar  
my_code = 34  
country = country_codes[my_code]  
print(country)
```

Spain

```
[34]: # Podem obtenir totes les claus:  
print(country_codes.keys())
```

dict_keys([34, 376, 41, 424])

```
[35]: # O els valors:  
print(country_codes.values())
```

dict_values(['Spain', 'Andorra', 'Switzerland', None])

```
[36]: # Podem modificar valors al diccionari o afegir noves claus.  
# Definim un diccionari buit.  
# 'country_codes = dict()' és una notació equivalent:  
country_codes = {}  
  
# Afegim un element:  
country_codes[34] = 'Spain'  
  
# N'afegim un altre:  
country_codes[81] = 'Japan'  
  
print(country_codes)
```

{34: 'Spain', 81: 'Japan'}

```
[37]: # Modifiquem el diccionari:  
country_codes[81] = 'Andorra'
```

```
print(country_codes)
```

```
{34: 'Spain', 81: 'Andorra'}
```

```
[38]: # Podem assignar el valor buit a un element:
country_codes[81] = None

print(country_codes)
```

```
{34: 'Spain', 81: None}
```

Els valors buits ens seran útils per declarar una variable que no sapiguem quin valor o quin tipus de valor contindrà i per fer comparacions entre variables. Habitualment, els valors buits són *None* o "", en el cas de les cadenes de caràcters.

```
[39]: # Podem assignar el valor d'una variable a una altra.
# És important que s'entenguin les línies següents:
a = 5
b = 1
print(a, b)
# b conté la 'direcció' del contenidor al qual apunta 'a'.
b = a
print(a, b)
```

```
5 1
5 5
```

```
[40]: # Vegem ara què passa si modifiquem el valor d'a o b:
a = 6
print(a, b)
b = 7
print(a, b)
```

```
6 5
6 7
```

Fins aquí hem presentat com declarar i utilitzar variables. Recomanem la lectura de la [documentació oficial en línia](#) per a fixar els coneixements explicats.

7 6. Organització del codi

Un mòdul de Python és qualsevol fitxer amb extensió *.py* que estigui sota la ruta del *path* de Python. El path de Python es pot consultar important la llibreria *sys*:

```
[41]: import sys
print(sys.path)
```

```
['/content', '/env/python', '/usr/lib/python37.zip', '/usr/lib/python3.7',  
'/usr/lib/python3.7/lib-dynload', '', '/usr/local/lib/python3.7/dist-packages',  
'/usr/lib/python3/dist-packages', '/usr/local/lib/python3.7/dist-  
packages/IPython/extensions', '/root/.ipython']
```

Per defecte, Python també mira les llibreries que s'hagin definit a la variable d'entorn \$PYTHON-PATH (això pot canviar lleugerament en un [entorn Windows](#)).

Un paquet en Python és qualsevol directori que contingui un fitxer especial de nom `__init__.py` (aquest fitxer estarà buit la majoria de vegades).

Un mòdul pot contenir diferents funcions, variables o objectes. Per exemple, definim un mòdul de nom `prog_datasci.py` que contingui:

```
[42]: # prog_datasci.py  
  
PI = 3.14159265  
  
def suma(x, y):  
    return x + y  
  
def resta(x, y):  
    return x - y
```

Per utilitzar des d'un altre mòdul o *script* aquestes funcions, hauríem d'escriure el següent:

```
[43]: # Especifiquem la ruta on tenim el fitxer .py  
  
from google.colab import drive  
drive.mount('/content/drive')  
%cd /content/drive/MyDrive/Colab_Notebooks/prog_datasci_2  
  
from prog_datasci import PI, suma, resta  
  
# I llavors podríem utilitzar-les normalment.  
rset = suma(2,5)
```

```
Mounted at /content/drive  
/content/drive/MyDrive/Colab_Notebooks/prog_datasci_2
```

També s'acostuma a utilitzar la paraula clau `as` per a donar un nou nom al mòdul o funcions que s'importen. Així, per exemple, podríem fer `from prog_datasci import PI as mypi` per fer servir el nom `mypi` per a referir-nos a `PI`:

```
[44]: from prog_datasci import PI as mypi  
print(mypi)
```

```
3.14159265
```

Durant el curs veurem que aquesta sintaxi és molt comuna per a escurçar els noms dels mòduls que importem.

A Python també podem utilitzar la directiva `from prog_datasci import *`, però el seu ús està **totalment desaconsellat**. La raó és que estariem important gran quantitat de codi que no utilitzarem (amb el consegüent augment de l'ús de la memòria), però a més, podríem tenir col·lisions de noms (funcions que es diguin de la mateixa manera en diferents mòduls) sense el nostre coneixement. Si no és que sigui imprescindible, no utilitzarem aquesta directiva i importarem una a una les llibreries i les funcions que necessitem.

Podeu aprendre més sobre importar llibreries i definir els vostres propis mòduls [aquí](#).

8 7. Operadors

Els operadors són símbols que indiquen a l'interpret de comandes que realitzi una operació específica. Hi ha diferents tipus segons la seva funció, que s'engloben en 5 blocs: operadors aritmètics, d'assignació, relacionals, lògics i de pertinença.

En aquest notebook ja hem anat veient operadors, però a continuació els descriurem més formalment i també veurem exemples de cada un d'ells:

8.1 7.1. Operadors aritmètics

Els operadors aritmètics són símbols que ens permeten realitzar una operació matemàtica bàsica. Per exemple, si considerem l'expressió `a=1+2`, tenim 1 i 2 com els operands i + com l'operador aritmètic.

Operador	Descripció	Exemple
+	Suma	$1 + 2 = 3$
-	Resta	$2 - 1 = 1$
	Multiplicació	$2 * 3 = 6$
/	Divisió	$5 / 2 = 2.5$
//	Divisió (nombre enter)	$5 // 2 = 2$
%	Mòdul	$16 \% 3 = 1$
	Potència	$2 ** 4 = 16$

```
[45]: # Definim dos elements: a i b
a = 2
b = 3

# Fem la suma d'a i b i mostrem el resultat
print(a + b)
```

5

8.2 7.2. Operadors d'assignació

Els operadors d'assignació s'utilitzen per especificar un valor a una variable. Un exemple ja comentat és l'operador `=`. Aquest operador `=` es pot combinar amb operadors aritmètics per a poder realitzar una operació i fer l'assignació alhora. Per exemple, utilitzem l'operador bàsic d'assignació `=` per a declarar una variable: `a = 5`. A continuació, utilitzem l'operador `+=` per a sumar 5 a la nostra variable `a` i actualitzar el seu valor: `a+=5`. Una altra manera d'escriure-ho és `a = a+5`.

```
[46]: # Definim un element a
a = 10
print(a)

# Fem la suma d'a i 5 i mostrem el resultat
a += 5
print(a)
```

10

15

Operador	Descripció	Alternativa
<code>=</code>	<code>a = 10</code> . Assignem el valor 10 a la variable <code>a</code>	—
<code>+=</code>	<code>a += 5</code> . Sumem 5 a la variable <code>a</code> i n'actualitzem el valor	<code>a = a + 5</code>
<code>-=</code>	<code>a -= 2</code> . Restem 2 a la variable <code>a</code> i n'actualitzem el valor	<code>a = a - 2</code>
<code>* =</code>	<code>a * = 2</code> . Multipliquem variable <code>a</code> per 2 i n'actualitzem el valor	<code>a = a * 2</code>
<code>/=</code>	<code>a /= 2</code> . Dividim la variable <code>a</code> entre 2 i n'actualitzem el valor	<code>a = a / 2</code>
<code>//=</code>	<code>a //= 2</code> . Dividim la variable <code>a</code> entre 2 (obtenint un enter) i n'actualitzem el valor	<code>a = a // 2</code>
<code>%=</code>	<code>a %= 3</code> . Calculem el mòdul d' <code>a</code> entre 3 i n'actualitzem el valor	<code>a = a % 3</code>
<code>** =</code>	<code>a ** = 2</code> . Elevem <code>a</code> al quadrat i n'actualitzem el valor	<code>a = a ** 2</code>

8.3 7.3. Operadors relacionals

Els operadors relacionals s'utilitzen per comparar dos variables segons una condició específica. Per exemple: *És la variable a més gran que la variable b?*

Aquests operadors retornen un tipus de dades concret, anomenat **boolean**, segons el resultat de comparació. Els **boolean** només poden tenir només 2 valors: **True** o **False**. Si la condició és certa, es retorna el boolean **True**. Si la condició és falsa, es retorna el boolean **False**.

Per exemple, considerem la situació següent:

Tenim dos variables, amb valor 2 i 4 i volem utilitzar els operadors **>** o **<** per comparar-les. En aquest cas, si escribim **2 > 4** estem preguntant si 2 és més gran que 4. Com aquesta condició és falsa, es retornarà el boolean **False**. En canvi, si fem **2 < 4**, la condició és certa, ja que 2 és més petit que 4, de manera que es retornarà el boolean **True**.

```
[47]: # Les operacions relacionals tindran com a resultat
      # un valor cert (True) o fals (False):
      a = 2
      b = 4

      # El valor a és més gran que b?
      print(a > b)
```

False

```
[48]: # El valor a és més petit que b?
      print(a < b)
```

True

```
[49]: # Definim un element b
      b = 5

      # El valor de b és igual que el d'a?
      print(b == a)

      # El valor de b és diferent que el d'a?
      print(b != a)
```

False

True

```
[50]: # Altres operadors relacionals disponibles són
      # més petit o igual '<=',
      # més gran o igual '>='
      print(a <= b)
      print(a >= b)
```

True
False

Operador	Descripció	Exemple
>	Retorna True si l'operand de l'esquerra és més gran que l'operand de la dreta	6 > 4 : True
<	Retorna True si l'operand de l'esquerra és més petit que l'operand de la dreta	2 < 4 : True
==	Retorna True si l'operand de l'esquerra és igual a l'operand de la dreta	4 == 4 : True
!=	Retorna True si l'operand de l'esquerra és diferent a l'operand de la dreta	2 != 4 : True
>=	Retorna True si l'operand de l'esquerra és més gran o igual que l'operand de la dreta	10 >= 5 : True
<=	Retorna True si l'operand de l'esquerra és més petit o igual que l'operand de la dreta	7 <= 7 : True

8.4 7.4. Operadors lògics

Els operadors lògics ens permeten treballar amb múltiples condicions. En Python en tenim 3: **and**, **or** i **not**.

Per exemple, considerem 2 elements:

a = 5; b = 2

I 2 condicions:

a > b (a més gran que b) ; a == b (a igual a b)

- Si considerem el primer operador lògic **and**, aquest retornarà True si es compleixen les 2 condicions. En aquest cas, la 1a condició (a > b) es compleix, però la 2a no (a == b). Per tant, (a > b) and (a == b) retornarà el Boolean False.
- En segon lloc, l'operador lògic **or** retornarà True si alguna de les condicions és certa. En aquest cas, com la primera condició es compleix, (a > b) or (a == b) retornarà el Boolean True.

- Finalment, l'operador lògic `not` retornarà `True` si la condició és falsa. Tal com hem comentat, la segona condició és falsa, aleshores, `not (a == b)` retornarà el Boolean `True`.

```
[51]: # Definim dos elements: a i b
a = 5
b = 2

# Mostrem per pantalla els resultats
print('Exemple 1: ', (a > b) and (a == b))
print('Exemple 2: ', (a > b) or (a == b))
print('Exemple 3: ', not (a == b))
```

Exemple 1: False

Exemple 2: True

Exemple 3: True

Operador	Descripció	Exemple
<code>and</code>	Retorna <code>True</code> si ambdues condicions són certes	<code>a and b</code>
<code>or</code>	Retorna <code>True</code> si alguna de les condicions és certa	<code>a or b</code>
<code>not</code>	Retorna <code>True</code> si la condició és falsa	<code>not a</code>

8.5 7.5. Operadors de pertinença

Els operadors de pertinença són molt utilitzats quan treballem amb llistes, ja que ens permeten comprovar si un element concret està dintre o no d'una llista. `in` i `not in` són els operadors de pertinença.

Considrem la llista següent: `fruites = ['pera', 'poma', 'platan']`

Volem comprovar que l'element `a` (`a = 'pera'`) està a la llista. Per fer-ho utilitzem l'operador `in`. En aquest cas, `a in fruites` retornarà el boolean `True`. Si agafem un altre element `b` (`b = 'kiwi'`), com aquest element no està a la llista, `not in` retornarà el boolean `True`, mentre que `in` retornarà el boolean `False`.

```
[52]: # Definim tres elements: fruites, a i b
fruites = ['pera', 'poma', 'platan']
a = 'pera'
b = 'kiwi'

# Mostrem el resultat de les operacions de pertinença
print(a in fruites) # pera està a la llista
print(b in fruites) # kiwi no està a la llista
print(b not in fruites) # kiwi no està a la llista
```

True

False

True

8.6 7.6. Numpy

Numpy és una llibreria de Python especialitzada en càlcul numèric. En aquesta unitat introduïrem alguns aspectes bàsics d'aquesta llibreria, però, més endavant, explicarem en detall com treballar amb aquesta llibreria i les seves funcionalitats més importants.

Si volem utilitzar la llibreria Numpy, primer de tot, l'hem d'importar amb la funció **import**, tal i com hem vist a l'apartat anterior. En el cas de Numpy, s'ha establert que la seva abreviació més comuna és **np**.

```
[53]: import numpy as np
```

Numpy ens permet fer operacions numèriques de manera molt senzilla. Per explicar al seva utilitat, considerarem la situació següent:

Tenim una llista amb el preu dels productes de la compra. Com podríem sumar tots els elements de la llista per obtenir el preu total?

Amb el que hem vist fins ara, una opció podria ser accedir a cada un dels elements de la llista i sumar-los amb l'operador aritmètic de la suma (+):

```
[54]: # Creem la llista
llista_compra = [3.5, 4.6, 8.9]

# Accedim a cada un dels elements de la llista i els sumem
suma_compra = llista_compra[0] + llista_compra[1] + llista_compra[2]

# Mostrem el resultat per pantalla
print('Suma: ', suma_compra)
```

Suma: 17.0

Tot i que el resultat és correcte, no sembla una solució gaire viable, ja que és molt manual i, si tenim molts elements dins la llista, no seria factible escriure'ls un a un.

Per solucionar aquest problema, podem utilitzar la llibreria Numpy.

Numpy conté multitud de funcions. En aquest cas, ens interessa sumar els elements d'una llista, per tant, utilitzarem la funció `sum()`.

```
[55]: # Creem la llista
llista_compra = [3.5, 4.6, 8.9]

# Apliquem la funció suma als elements de la llista
# i mostrem el resultat per pantalla
print('Suma: ', np.sum(llista_compra))
```

Suma: 17.0

Com podeu veure, amb Numpy hem pogut sumar tots els elements de la llista de manera fàcil i ràpida.

Numpy també es pot utilitzar per a calcular variables estadístiques (mitjana o mediana, desviació estàndard, màxim, mínim, entre d'altres) dels elements d'una llista:

```
[56]: # Apliquem la funció mean (mitjana)
# als elements de la llista
# i mostrem el resultat per pantalla
print('Mitjana: ', np.mean(llista_compra))

# Apliquem la funció median (mediana)
# als elements de la llista
# i mostrem el resultat per pantalla
print('Mediana: ', np.median(llista_compra))

# Apliquem la funció std (desviació estàndard)
# als elements de la llista
# i mostrem el resultat per pantalla
print('Desviació estàndard: ', np.std(llista_compra))

# Apliquem la funció max (màxim)
# als elements de la llista
# i mostrem el resultat per pantalla
print('Màxim: ', np.max(llista_compra))

# Apliquem la funció min (mínim)
# als elements de la llista
# i mostrem el resultat per pantalla
print('Mínim estàndard: ', np.min(llista_compra))
```

```
Mitjana:  5.666666666666667
Mediana:  4.6
Desviació estàndard:  2.3299976156401727
Màxim:    8.9
Mínim estàndard:    3.5
```

En aquest [enllaç](#) podeu consultar les operacions matemàtiques que conté la llibreria Numpy.

9 8. Recursos per al programador de Python

Part del procés d'aprendre a programar en Python consisteix a **aprendre on i com buscar** informació per resoldre problemes que no sabem solucionar immediatament. Així, familiaritzar-nos amb la documentació oficial, aprendre a fer servir els buscadors (com ara Google) per ajudar-nos a trobar la informació que necessitem, i descobrir el funcionament de les eines de pregunta-i-resposta (question-and-answer o Q&A) que fan servir els programadors, seran competències a treballar en l'assignatura. Aquestes competències són de vital importància, ja que els llenguatges de programació estan en contínua evolució, de manera que per molt que coneguem un llenguatge, és habitual que sovint ens trobem amb la necessitat de cercar algun recurs extern que ens ajudi a dur a terme les tasques que necessitem realitzar.

Durant el curs, anirem veient com consultar aquests recursos, i els utilitzarem per tal de resoldre les activitats que s'aniran plantejant. Veureu que els enunciats de les activitats que us proposem realitzar contenen indicacions sobre la consulta de recursos externs. Aquestes indicacions us aniran guiant en la cerca d'informació per tal de resoldre els exercicis. Al principi del curs, les indicacions seran molt explícites, i conforme el curs va avançant ho deixaran de ser progressivament. D'aquesta manera, anireu adquirint autonomia en el procés de búsqueda d'informació, cosa que us servirà per afrontar els problemes de programació que trobeu en el futur! Ara bé, recordeu que durant tot el procés esteu sempre acompanyats pel consultor de la vostra aula, que us ajudarà en tot moment (no dubteu a preguntar tot allò que calgui!).

10 9. Exercicis i preguntes teòriques

La part avaluable d'aquesta unitat consisteix en el lliurament d'un fitxer IPython Notebook amb extensió IPYNB que contindrà els diferents exercicis i les preguntes teòriques que s'han de contestar. Trobareu el fitxer (`prog_datasci_2_python_entrega.ipynb`) amb les activitats a la mateixa carpeta que aquest notebook que esteu llegint.

10.1 9.1. Instruccions importants

És molt important que a l'hora de lliurar el fitxer Notebook amb les vostres activitats us assegureu que:

1. Les vostres solucions siguin originals. Esperem no detectar-hi còpia directa entre estudiants.
2. Tot el codi estigui correctament documentat. El codi sense documentar equivaldrà a un 0.
3. El fitxer comprimit que lliureu és correcte (conté les activitats de la PAC que heu de lliurar).

Per fer el lliurament, heu d'anar a la carpeta del drive Colab Notebooks, clicant amb el botó dret a la PAC en qüestió i fent Download. D'aquesta manera, us baixareu la carpeta de la PAC comprimida amb zip. Aquest és el fitxer que heu de pujar al campus virtual de l'assignatura.

11 10. Bibliografia

D'una banda, us recomanem consultar la [introducció a Jupyter Notebook](#), per familiaritzar-vos amb l'entorn de treball.

D'altra banda, per acabar d'entendre els conceptes sobre variables presentats en aquesta unitat, us recomanem consultar el [vídeotutorial sobre tipus de variable](#) (podeu descarregar-lo per veure'l amb millor qualitat) i el [vídeotutorial sobre strings i llistes](#).